14.2.2 Storing sensitive information with Secrets (or not)

The most critical part of configuring applications is managing secret information like passwords, certificates, tokens, and keys. Kubernetes provides a Secret object to hold such data and pass it to containers.

A *Secret* is an API object used to store and manage sensitive information, such as passwords, OAuth tokens, and ssh keys. Pods can consume Secrets as environment variables or configuration files in a volume (https://kubernetes.io/docs/concepts/configuration/secret).

What makes this object *secret* is the process used to manage it. By themselves, Secrets are just like ConfigMaps. The only difference is that data in a Secret is usually Base64-encoded, a technical choice made to support binary files. Any Base64-encoded object can be decoded in a very straightforward way. It's a common mistake to think that Base64 is a kind of encryption. If you remember only one thing about Secrets, make it the following: *Secrets are not secret!*

The configuration we have been using to run Polar Bookshop on a local Kubernetes cluster relies on the same default credentials used in development, so we won't need Secrets yet. We'll start using them in the next chapter when deploying applications in production. For now, I want to show you how to create Secrets. Then I'll go through some options you have for ensuring that they are adequately protected.

One way of creating a Secret is using the Kubernetes CLI with an imperative approach. Open a Terminal window and generate a `test-credentials` Secret object for some fictitious test credentials (`user`/`password`).